스프린트 사이클 실행

사용법

명령어	동작
`/sprint [화면명]`	지정한 화면 1개에 대해 스프린트 1회 실행
`/sprint all`	전체 화면을 순차적으로 자동 실행 (모두 끝날 때까지)

/sprint all — 전체 자동 실행

전체 화면을 MVP 우선순위 순서대로, 각 화면마다 Planner → Generator → Evaluator 사이클을 돌며 PASS할 때까지 반복합니다. PASS하면 다음 스프린트로 자동 진행합니다.

실행 순서

```
Sprint 01: 프로젝트 초기 셋업 (AppColors, AppTypography, AppSpacing,
DatabaseHelper, 테마)
Sprint 02: 온보딩 (웰컴 → 아기 등록 → 주차 소개 → 알림 권한)
Sprint 03: 홈 화면 (대시보드)
Sprint 04: 기록 탭 (데일리 기록 + 성장 곡선 + 히스토리)
Sprint 05: 활동 탭 (활동 목록 + 활동 상세 + 타이머)
Sprint 06: 활동 후 관찰 기록 (관찰 폼 + 결과)
Sprint 07: 발달 체크 탭 (체크리스트 + 결과 + 추이)
Sprint 08: 마이 탭 (프로필 + 설정)
Sprint 09: 로컬 알림 (데일리 미션, 주차 전환, 마일스톤)
Sprint 10: 콘텐츠 시드 데이터 (0-1주, 2-3주 활동/체크리스트)
```

실행 흐름

```
Sprint 01 시작
  │
  ├── Planner → 사양서 작성 → 자동 승인
  │
  ├── Generator → 구현
  │
  ├── Evaluator → 검증
  │     │
  │     ├── PASS ──────────────────────────→ Sprint 02로 자동 진행
  │     │
  │     └── FAIL → Generator (수정) → Evaluator (재검증)
  │                                     │
  │                                     ├── PASS → Sprint 02로 자동 진행
  │                                     │
  │                                     └── FAIL → Generator (재수정) →
```

```
Evaluator
  │                                                                    │
  │                                                                    └── 최대
3회 후
  │                                                                        FAIL
이슈 기록,
  │
Sprint 02로 진행
  │
Sprint 02 시작
  ├── (동일 흐름)
  ...
Sprint 10 완료
  └── 최종 리포트 출력
```

사용자 개입

사양서는 **자동 승인**됩니다. Planner가 작성 완료하면 즉시 Generator로 진행합니다.
사용자가 "중지"를 입력하면 전체 실행을 중단합니다.

진행 상황 표시

각 스프린트 완료 시:

```
────────────────────────────────
  전체 진행: ████████▒▒▒▒▒▒▒▒ 5/10

  ☑ Sprint 01: 초기 셋업         PASS
  ☑ Sprint 02: 온보딩            PASS
  ☑ Sprint 03: 홈 화면           PASS (2회 시도)
  ☑ Sprint 04: 기록 탭           PASS
  ⟳ Sprint 05: 활동 탭           진행 중...
  ☐ Sprint 06: 관찰 기록
  ☐ Sprint 07: 발달 체크
  ☐ Sprint 08: 마이 탭
  ☐ Sprint 09: 로컬 알림
  ☐ Sprint 10: 시드 데이터
────────────────────────────────
```

최종 리포트

모든 스프린트 완료 후 `sprints/sprint-final-report.md`를 생성합니다:

```
# 전체 스프린트 완료 리포트

## 요약
- 전체 스프린트: 10개
- PASS: 9개
```

```
- PARTIAL (수동 수정 필요): 1개
- 총 시도 횟수: 14회 (평균 1.4회/스프린트)

## 스프린트별 결과
| # | 대상 | 판정 | 시도 횟수 | 생성 파일 수 |
|---|---|---|---|---|
| 01 | 초기 셋업 | PASS | 1회 | 8개 |
| 02 | 온보딩 | PASS | 1회 | 6개 |
| ... | ... | ... | ... | ... |

## 미해결 이슈
- Sprint 07 FAIL-2: 레이더 차트 다크 모드 미대응 (수동 수정 필요)

## 생성된 전체 파일 목록
- lib/core/theme/app_colors.dart
- lib/core/theme/app_typography.dart
- ...
```

/sprint [화면명] — 단일 스프린트 실행

지정한 화면/기능에 대해 스프린트 1회를 실행합니다.

실행 흐름

```
Step 1: Planner → 스프린트 사양서 작성
Step 2: Generator → Flutter 코드 구현
Step 3: Evaluator → 검증 및 PASS/FAIL 판정
Step 4: (FAIL 시) Generator → 피드백 기반 재구현
Step 5: (재구현 시) Evaluator → 재검증
반복... (최대 3회)
```

Step 1: Planner 호출

Planner sub-agent를 호출하여 스프린트 사양서를 작성합니다.

프롬프트:

```
다음 화면/기능에 대한 스프린트 사양서를 작성해주세요: {사용자가 지정한 대상}

참조할 문서:
- docs/2_서비스기획서.md
- docs/2_개발기획서.md
- docs/3_디자인컴포넌트.md

sprints/ 폴더에 사양서를 저장해주세요.
```

Planner가 완료하면 `sprints/sprint-{번호}-{화면명}.md` 파일이 생성됩니Planner 완료 즉시 Generator로 자동 진행합니다.

Step 2: Generator 호출

Generator sub-agent를 호출하여 코드를 구현합니다.

프롬프트:

```
스프린트 사양서를 읽고 Flutter 코드를 구현해주세요.

사양서: sprints/sprint-{번호}-{화면명}.md
참조: docs/2_개발기획서.md, docs/3_디자인컴포넌트.md

구현 완료 후 sprints/sprint-{번호}-result.md에 결과를 기록해주세요.
```

Generator가 완료하면:

- `lib/` 하위에 Dart 파일들이 생성됩니다.
- `sprints/sprint-{번호}-result.md`에 구현 결과가 기록됩니다.

Step 3: Evaluator 호출

Evaluator sub-agent를 호출하여 검증합니다.

프롬프트:

```
Generator가 구현한 코드를 스프린트 사양서 기준으로 검증해주세요.

사양서: sprints/sprint-{번호}-{화면명}.md
구현 결과: sprints/sprint-{번호}-result.md
검증할 코드: Generator가 생성한 파일들

검증 결과를 sprints/sprint-{번호}-review.md에 작성해주세요.
```

Step 4: 결과 처리

Evaluator의 판정 결과에 따라:

PASS인 경우:

```
☑ Sprint {번호} 완료!

검증 결과: PASS
생성된 파일: {파일 목록}
```

사용자에게 결과를 알리고 스프린트를 종료합니다.

FAIL 또는 PARTIAL인 경우:

```
⚠ Sprint {번호} 검증 실패

FAIL 항목:
{FAIL 항목 요약}

Generator에게 피드백을 전달하고 재구현을 시작합니다. (시도 {N}/3)
```

Generator를 다시 호출합니다:

```
Evaluator 검증 결과 FAIL 항목이 있습니다. 피드백을 읽고 수정해주세요.

피드백: sprints/sprint-{번호}-review.md
수정 후 sprints/sprint-{번호}-result.md를 업데이트해주세요.
```

수정 후 Evaluator를 다시 호출하여 재검증합니다.

최대 반복 횟수

Generator ↔ Evaluator 반복은 **최대 3회**입니다.
3회 후에도 FAIL이면 사용자에게 남은 이슈를 보고하고 수동 개입을 요청합니다.

스프린트 번호 관리

`sprints/` 폴더의 기존 파일을 확인하여 다음 번호를 자동 부여합니다.
기존 파일이 없으면 1번부터 시작합니다.

파일 구조 예시

```
sprints/
├── sprint-01-setup.md              # Planner 작성 (사양서)
├── sprint-01-result.md             # Generator 작성 (구현 결과)
├── sprint-01-review.md             # Evaluator 작성 (검증 결과)
├── sprint-02-onboarding.md
├── sprint-02-result.md
├── sprint-02-review.md
├── ...
└── sprint-final-report.md          # 전체 완료 시 최종 리포트
```